Cheatsheet · 21 chapters on one page

Git and CI/CD cheatsheet

Every command, key idea, table, memory aid and common mistake from the chapters, gathered in one place for quick revision. Nothing here is new. Each card links back to the lesson that teaches it, so open the lesson when a line does not make sense yet.

100 key ideas | 101 command cards | 55 tables | 53 tips and traps

STEP 1 Getting started with Git

1.1 What Git is and how to set it up

Why version control exists, centralised versus distributed, snapshots, installing Git, git config levels and a first repository.

KEY IDEAS

- **Version control** Records every version of your files with author, time and message, so you can compare, restore and work in parallel.
- **Distributed** Every Git clone has the full history. Commit, log and diff work offline. Centralised tools need the server.
- **Snapshots** A commit is a snapshot of the whole project. Unchanged files are reused, not stored again.
- **Config levels** system, global (~/.gitconfig), local (.git/config). Later wins. Set user.name, user.email and init.defaultBranch first.
- **Git is not GitHub** Git is the tool. GitHub, GitLab and Bitbucket host repositories and add pull requests, reviews and CI/CD.

COMMANDS

Set a value for your user — Exam favourite

```
git config --global <key> "<value>"
```

Stored in ~/.gitconfig. Use --local (the default inside a repository) to set it for one repository only.

Lesson: Install Git and tell it who you are

Which value wins — Exam favourite

system < global < local

Git reads /etc/gitconfig, then ~/.gitconfig, then .git/config. A later file overrides an earlier one, so the most specific setting wins.

Lesson: Install Git and tell it who you are

Get help

`git help <command>` or `git <command> -h`

The first opens the full manual page, the second prints a short list of options in the terminal.

Lesson: Install Git and tell it who you are

TABLES

Table: problem without version control · what version control gives you

Problem without version control	What version control gives you
Many copies with confusing names	One folder, a clean history of saved snapshots
No idea what changed	An exact list of changed lines between any two versions
A mistake ruins the only copy	Go back to any earlier version
Two people overwrite each other	Separate branches that are merged with checks
Nobody knows why a line exists	Every change carries an author, a date and a message

Lesson: Why version control exists

Table: kind · examples · where the history lives · weak spot

Kind	Examples	Where the history lives	Weak spot
Local	RCS	One computer	No teamwork, one disk failure loses everything
Centralised	CVS, Subversion (SVN), Perforce	One central server	Server down means nobody can commit or read history
Distributed	Git, Mercurial	Every clone has the full history	More ideas to learn at the start

Lesson: Where Git came from and how it is different

Table: system · how to install

System	How to install
Windows	Download Git for Windows from git-scm.com. It includes Git Bash, a terminal where every command on these pages works.
macOS	Run `git --version`. If Git is missing, macOS offers to install the Xcode command line tools. Homebrew users can run `brew install git`.
Linux (Debian, Ubuntu)	`sudo apt install git`
Linux (Fedora)	`sudo dnf install git`

Lesson: Install Git and tell it who you are

TIPS AND TRAPS

REMEMBER init.defaultBranch decides the name of the first branch in new repositories. Git 2.28 added this setting. If it is not set, current Git still uses master and prints a hint, and the Git project has said that new repositories will use main by default in Git 3.0.

Lesson: Install Git and tell it who you are

REMEMBER Using a work laptop for personal projects? Set your personal email globally and your work email with git config user.email inside each work repository. The local value wins there.

Lesson: Install Git and tell it who you are

COMMON MISTAKE Running git init in the wrong folder, for example your whole home folder, makes Git track everything. If that happens, delete the new .git folder in that place. Your files are not touched.

Lesson: Your first repository in two minutes

IN THE EXAM Git and GitHub are not the same thing. Git is the tool on your computer. GitHub, GitLab and Bitbucket are websites that host Git repositories and add pull requests, issues and CI/CD pipelines.

Lesson: From intuition to research

1.2 The three areas: working tree, staging area and repository

Working tree, staging area and repository, file states and status codes, add, add -p, commit and good commit messages.

KEY IDEAS

- **Three areas** Working tree (your files), staging area or index (next snapshot), repository (all commits). add moves changes to the index, commit saves the index.
- **Status codes** git status -s prints XY: X staged (index vs HEAD), Y unstaged (folder vs index). ?? untracked, MM staged and changed again.
- **Commit saves the index** Edits made after git add are not in the commit until you add again. commit -a stages tracked files only.
- **Partial staging** git add -p stages hunk by hunk so one file can feed two clean commits.
- **Good messages** Short imperative summary near 50 characters, blank line, then why. One logical change per commit.

COMMANDS

Short status Exam favourite

`git status -s` → `XY path`

X compares the index with HEAD (what is staged). Y compares the working tree with the index (what is not staged yet). ?? means untracked.

Lesson: File states and git status

Long status

`git status`

Lists staged changes, unstaged changes and untracked files, with the command to undo each.

Lesson: File states and git status

Stage Exam favourite

`git add <file>`, `git add .`, `git add -p`

git add . stages everything under the current folder. -p asks about each changed piece (hunk) so you can stage part of a file.

Lesson: Adding and committing well

Commit Exam favourite

`git commit -m "message"`

Saves the staging area. Without -m Git opens your editor for a longer message.

Lesson: Adding and committing well

Stage tracked files and commit

`git commit -am "message"`

-a stages every change to tracked files first. New untracked files are not included.

Lesson: Adding and committing well

Remove and rename

`git rm <file>`, `git mv <old> <new>`

Changes the folder and stages the change in one step. git rm --cached keeps the file on disk but stops tracking it.

Lesson: Adding and committing well

TABLES

Table: area · also called · what it holds · studio picture

Area	Also called	What it holds	Studio picture
Working tree	working directory	The files you see and edit in your folder	the studio floor
Staging area	the index	The exact snapshot your next commit will save	the stage
Repository	.git directory, HEAD	Every commit ever made	the album

Lesson: The three areas of a Git project

Table: state · meaning · how it gets there · short code

State	Meaning	How it gets there	Short code
Untracked	New file Git is not watching	You create it	`??`
Unmodified	Same as the last commit	After a commit or checkout	(not listed)
Modified	Changed, not staged	You edit a tracked file	`M`
Staged	Change is in the index	git add	`M` or `A`
Staged and modified again	Index has one version, the folder a newer one	edit after git add	`MM`

Lesson: File states and git status

Table: rule · good · bad

Rule	Good	Bad
Short summary line, about 50 characters	Fix crash when the password is empty	fixed stuff
Imperative mood, as if giving an order	Add search by price	Added search, adding search
Blank line, then the why in the body	Users saw a 500 error on empty input...	(nothing)
One logical change per commit	One bug fix	Bug fix plus renaming plus new feature

Lesson: Adding and committing well

TIPS AND TRAPS

COMMON MISTAKE git commit saves what is staged, not what is in your file. If you edit a file after git add, the newer edit is not in the commit until you add it again.

Lesson: The three areas of a Git project

REMEMBER Conventional Commits is a popular format: feat: add search by price, fix: handle empty password, docs: update README. Tools read it to make changelogs and choose the next version number.

Lesson: Adding and committing well

1.3 Reading history: log, diff, blame and naming commits

git log filters and the pickaxe, the three diffs and hunk headers, ~, ^, reflog names, .. and ... ranges, and git blame.

KEY IDEAS

- **git log** --oneline --graph --all for the picture. -S text to find when code appeared. -p and --stat for what changed. -- path for one file.
- **Three diffs** git diff: folder vs index. git diff --staged: index vs HEAD. git diff HEAD: folder vs HEAD.
- **Hunks** @@ -a,b +c,d @@: old lines a to a+b-1 became new lines c to c+d-1. A changed line is one - and one +.
- **Naming commits** X~n: n first parents back. X^n: n-th parent of a merge. HEAD@{n}: where HEAD was n moves ago.
- **Ranges** A..B: in B not in A. A...B: in one but not both (log), or merge base to B (diff).

COMMANDS

Compact history Exam favourite

```
git log --oneline --graph --
    decorate --all
```

One line per commit, a text drawing of branches and merges, branch and tag names, every branch not just the current one.

Lesson: Reading history with git log

Filter

```
git log -n 5, --author=Asha, --
since="2 weeks ago", --grep=login
```

Last 5 commits, by one person, in a time window, or with a word in the message.

Lesson: Reading history with git log

What a change touched

`git log -p`, `git log --stat`, `git log -- path/file`

-p adds the full diff, --stat lists changed files with counts, and -- path limits history to one file.

Lesson: Reading history with git log

Find code

Exam favourite

`git log -S "price_limit"`

The pickaxe: commits that added or removed that text. Great for "when did this function appear?".

Lesson: Reading history with git log

One commit in full

`git show <commit>`

Message plus diff. git show HEAD:app.py prints the file as it was in that commit.

Lesson: Reading history with git log

Hunk header

Exam favourite

`@@ -a,b +c,d @@`

The old file from line a for b lines became the new file from line c for d lines. Lines start with space (context), - (removed) or + (added).

Lesson: git diff: three comparisons and how to read a hunk

Word level and stats

`git diff --word-diff`, `git diff --stat`

Shows changed words inside a line, or only counts per file.

Lesson: git diff: three comparisons and how to read a hunk

Ancestors

Exam favourite

`X~n` = n first parents back. `X^n` = the n-th parent of X

HEAD~1 is the parent, HEAD~2 the grandparent. For a merge commit, X^1 is the branch you were on and X^2 is the branch you merged in.

Lesson: Naming commits: HEAD, ~, ^ and ranges

Two dots

Exam favourite

`A..B` = reachable from B but not from A

git log main..feature lists what feature has that main does not.

Lesson: Naming commits: HEAD, ~, ^ and ranges

Three dots

`A...B` = reachable from either but not both

With git log --left-right each commit is marked < or > for its side.

Lesson: Naming commits: HEAD, ~, ^ and ranges

Where HEAD was

`HEAD@{n}`

The n-th previous position of HEAD, from the reflog. HEAD@{1} is where you were before the last move.

Lesson: Naming commits: HEAD, ~, ^ and ranges

Line by line authors

`git blame -L 10,20 app.py`

For each line: the commit, author and date that last changed it. -w ignores whitespace changes, -C finds lines moved from other files.

Lesson: Who wrote this line and why

Ignore big reformat commits

`git blame --ignore-revs-file .git-blame-ignore-revs`

Skip commits listed in the file, such as a project wide formatting run, so blame shows the real author. GitHub reads this file too.

Lesson: Who wrote this line and why

TABLES

Table: command · compares · answers

Command	Compares	Answers
`git diff`	working tree with the index	What have I changed but not staged?
`git diff --staged`	index with HEAD	What will my next commit contain?
`git diff HEAD`	working tree with HEAD	Everything since the last commit
`git diff main feature`	two commits	How do these branches differ?
`git diff main...feature`	merge base of the two with feature	What did feature add since it split off?

Lesson: git diff: three comparisons and how to read a hunk

TIPS AND TRAPS

COMMON MISTAKE git diff with no arguments shows nothing after git add. The changes did not vanish: they moved to the index. Use git diff --staged.

Lesson: git diff: three comparisons and how to read a hunk

IN THE EXAM HEAD~2 and HEAD^^ name the same commit. HEAD^2 is different: it is the second parent of HEAD and only exists if HEAD is a merge commit.

Lesson: Naming commits: HEAD, ~, ^ and ranges

STEP 2 Branching and merging

2.1 Branches and HEAD

Branches as pointers, HEAD, switch and checkout, detached HEAD and rescue, renaming and deleting branches.

KEY IDEAS

- **Branch** A branch is a ref, a 41 byte file with a commit id. Creating one copies nothing. Commits move only the branch HEAD points to.
- **switch** git switch name, git switch -c name, git switch - (previous). checkout does the same in older guides.
- **Detached HEAD** HEAD points at a commit. Commits made there need git switch -c name, or they are left for the reflog only.
- **Delete** git branch -d refuses unmerged work. -D forces. --merged and --no-merged help clean up.
- **Protect work** Git refuses to switch if uncommitted edits would be overwritten. Commit or stash first.

COMMANDS

Create

```
git branch <name> [<start>]
```

Makes a new pointer at HEAD, or at start. You stay where you are.

Lesson: A branch is just a movable label

Switch Exam favourite

```
git switch <name>
```

Moves HEAD to that branch and updates your files to its snapshot.

Lesson: A branch is just a movable label

Create and switch Exam favourite

```
git switch -c <name>
```

The same as the older git checkout -b <name>.

Lesson: A branch is just a movable label

See branches

```
git branch -vv, git branch -a
```

-vv shows the tip commit and the upstream of each branch, -a adds remote tracking branches.

Lesson: A branch is just a movable label

Rename

```
git branch -m <old> <new>
```

With one name it renames the current branch.

Lesson: Naming, renaming and deleting branches

Delete safely Exam favourite

```
git branch -d <name>
```

Refuses if the branch has commits not merged into the current branch or its upstream.

Lesson: Naming, renaming and deleting branches

Force delete

```
git branch -D <name>
```

Deletes even unmerged work. The commits survive in the reflog for a while.

Lesson: Naming, renaming and deleting branches

Merged or not

```
git branch --merged, --no-merged
```

Lists branches whose tips are (or are not) contained in HEAD. Good for clean up.

Lesson: Naming, renaming and deleting branches

TABLES

Table: job · newer command · older command

Job	Newer command	Older command
Go to a branch	`git switch feature`	`git checkout feature`
Create a branch and go to it	`git switch -c feature`	`git checkout -b feature`
Go back to the previous branch	`git switch -`	`git checkout -`
Look at an old commit	`git switch --detach a1b2c3d`	`git checkout a1b2c3d`
Throw away edits in a file	`git restore app.py`	`git checkout -- app.py`

Lesson: switch, checkout and what happens to your files

Table: naming habit · example

Naming habit	Example
type and short description	feature/search-by-price, fix/login-crash
ticket number	feature/SHOP-142-search
no spaces, lowercase, dashes	release/2.4

Lesson: Naming, renaming and deleting branches

TIPS AND TRAPS

REMEMBER Only the branch HEAD points to moves when you commit. Other branches stay exactly where they are. That is the whole trick behind parallel work.

Lesson: A branch is just a movable label

COMMON MISTAKE Commit or stash (chapter 3.2) before switching when Git complains. Never fix it with a hard reset unless you really want to lose the edits.

Lesson: switch, checkout and what happens to your files

REMEMBER If you already left the detached commits, run git reflog, find the id, and run git branch rescue <id>.

Lesson: Detached HEAD, without fear

2.2 Merging and resolving conflicts

Fast-forward versus three way merge, merge base, squash, conflicts and markers, abort, diff3, rerere and the ort strategy.

KEY IDEAS

- **Fast-forward** If the current branch has not moved, merge just moves the pointer. --no-ff forces a merge commit, --ff-only forbids one.
- **Three way merge** Base, ours, theirs. One side changed: take it. Same change: take it. Different changes: conflict.
- **Resolve** Edit, delete <<<<<<< ======= >>>>>>> markers, git add, git commit. git merge --abort to give up.
- **Strategies** ort is the default since Git 2.34. -s ours keeps our tree, -X ours only settles conflicts our way.
- **Clean is not correct** A merge without conflicts can still break the program. Test the merged result in CI.

COMMANDS

Merge a branch into the current one

Exam favourite

```
git merge <branch>
```

Fast-forwards when it can, otherwise makes a merge commit.

Lesson: Fast-forward or a real merge

Always make a merge commit

```
git merge --no-ff <branch>
```

Keeps the feature visible as its own bubble in history, even when a fast-forward was possible.

Lesson: Fast-forward or a real merge

Refuse anything but a fast-forward

```
git merge --ff-only <branch>
```

Fails instead of making a merge commit. Safe default for updating a branch you do not work on.

Lesson: Fast-forward or a real merge

Squash

`git merge --squash <branch>` then `git commit`

Stages all changes of the branch as one new ordinary commit. The branch itself is not recorded as merged.

Lesson: Fast-forward or a real merge

Merge base

Exam favourite

```
git merge-base main feature
```

Prints the best common ancestor that a three way merge uses as its base.

Lesson: Fast-forward or a real merge

Abort

Exam favourite

```
git merge --abort
```

Puts everything back as it was before git merge started.

Lesson: Resolving a conflict step by step

Take one whole side for a file

`git checkout --ours file` or `--theirs file`

Then git add file. Useful for generated files such as lock files.

Lesson: Resolving a conflict step by step

See the base too

```
git config merge.conflictStyle diff3
```

Markers then also show the base version between ||||||| and =======. zdiff3 is a tidier variant.

Lesson: Resolving a conflict step by step

Remember resolutions

```
git config rerere.enabled true
```

Reuse recorded resolution: when the same conflict comes back, Git applies your earlier fix.

Lesson: Resolving a conflict step by step

TABLES

Table: situation · what git does · new commit?

Situation	What Git does	New commit?
main has not moved since feature started	**Fast-forward**: main just slides up to the tip of feature	No
Both branches have new commits	**Three way merge**: compares both tips with their merge base and makes a merge commit with two parents	Yes
feature is already contained in main	Already up to date	No

Lesson: Fast-forward or a real merge

Table: base · ours · theirs · result

Base	Ours	Theirs	Result
x	x	x	x (nobody changed it)
x	y	x	y (only we changed it)
x	x	z	z (only they changed it)
x	y	y	y (we both made the same change)
x	y	z	**CONFLICT**, you decide

Lesson: How a three way merge decides

TIPS AND TRAPS

COMMON MISTAKE A file with conflict markers still compiles in some languages, for example inside a YAML string or a Markdown file. Search for <<<<<<< before committing, or let a pre-commit check do it.

Lesson: Resolving a conflict step by step

IN THE EXAM A clean merge is not a correct merge. Two changes can merge without any conflict and still break the program together. That is why CI on pull requests tests the merged code, not only the branch.

Lesson: From intuition to research

2.3 Rebase, interactive rebase and cherry-pick

Rebase, the golden rule, merge versus rebase, interactive rebase (pick, squash, fixup, drop) and cherry-pick.

KEY IDEAS

- **Rebase** Replays your commits on a new base. Copies get new ids. --continue after fixing, --abort to give up.
- **Golden rule** Never rebase commits others may have built on. Rebase your own branch, then push --force-with-lease.
- **Merge or rebase** Merge keeps true history and is safe on shared branches. Rebase gives a straight line for your own work.
- **Interactive rebase** rebase -i: pick, reword, edit, squash, fixup, drop, reorder. commit --fixup plus --autosquash automates it.
- **Cherry-pick** Copies one commit as a new commit, for example a fix to a release branch. -x records the source.

COMMANDS

Rebase the current branch

Exam favourite

```
git rebase <upstream>
```

Replays the commits of HEAD that are not in upstream on top of upstream.

Lesson: Rebase: move your work onto a new base

Continue, skip or stop

Exam favourite

```
git rebase --continue, --skip, --abort
```

After fixing a conflict: git add, then --continue. --abort returns to before the rebase.

Lesson: Rebase: move your work onto a new base

Move a range somewhere else

```
git rebase --onto <newbase> <oldbase> [<branch>]
```

Takes the commits after oldbase and replays them on newbase.

Lesson: Rebase: move your work onto a new base

Update stacked branches too

```
git rebase --update-refs
```

Also moves other branches that point at commits being rewritten (Git 2.38 and later).

Lesson: Rebase: move your work onto a new base

Prepare a fixup now, squash later

Exam favourite

`git commit --fixup=<commit>` then `git rebase -i --autosquash <base>`

The fixup! commit is moved next to its target and marked fixup automatically.

Lesson: Interactive rebase: tidy your commits before review

Copy commits

Exam favourite

`git cherry-pick <commit>...`, `git cherry-pick A..B`

Each picked commit becomes a new commit with a new id. -x adds "(cherry picked from commit ...)" to the message.

Lesson: Cherry-pick: copy one commit

Undo a commit by adding its opposite

```
git revert <commit>
```

Covered in chapter 3.1. Safe on shared branches because nothing is rewritten.

Lesson: Cherry-pick: copy one commit

TABLES

Table: · merge · rebase

	Merge	Rebase
History	True record, with merge commits	Clean straight line, rewritten
Commit ids	Unchanged	New ids for replayed commits
Safe on shared branches	Yes	No, only on your own branch
Conflicts	Solved once, in the merge commit	May be solved once per replayed commit
Good for	Bringing a finished feature into main	Updating your feature branch with main before review

Lesson: The golden rule, and merge versus rebase

Table: command · effect

Command	Effect
pick	keep the commit as it is
reword	keep the change, edit the message
edit	stop after this commit so you can amend it or split it
squash	meld into the previous commit and combine the messages
fixup	meld into the previous commit and throw this message away
drop	remove the commit
(change the order of lines)	reorder the commits

Lesson: Interactive rebase: tidy your commits before review

TIPS AND TRAPS

COMMON MISTAKE Rebased commits are copies with new ids, because their parent changed. Anyone who had the old ones now has commits that no longer exist on the branch.

Lesson: Rebase: move your work onto a new base

COMMON MISTAKE After rebasing a branch you already pushed, a normal push is rejected. Use --force-with-lease, never plain --force, and only on a branch that is yours. Chapter 4.1 explains why.

Lesson: The golden rule, and merge versus rebase

REMEMBER Interactive rebase rewrites history. Use it on commits that only you have, usually right before opening or updating a pull request.

Lesson: Interactive rebase: tidy your commits before review

STEP 3 Undo, stash, tags and ignore rules

3.1 Undoing things: restore, reset, revert and the reflog

restore, amend, the three reset modes, revert, reflog rescue and when each undo is safe.

KEY IDEAS

- **Where is it** Unstaged edit: git restore file. Staged: git restore --staged file. Local commit: reset or amend. Shared commit: revert.
- **Reset modes** --soft moves the branch only. --mixed (default) also resets the index. --hard also resets your files.
- **Revert** Makes a new inverse commit. Safe on shared branches. Merge commits need -m 1.
- **Reflog** Local diary of HEAD. git reflog, then git reset --hard HEAD@{n} or git branch rescue . Unreachable entries expire after 30 days by default.
- **Danger** reset --hard and restore throw away uncommitted edits, which no command can bring back.

COMMANDS

Undo the last commit, keep the work staged

Exam favourite

```
git reset --soft HEAD~1
```

Then git commit again, for example to combine two commits.

Lesson: git reset: soft, mixed and hard

Undo the last commit, keep the work unstaged

```
git reset HEAD~1
```

Same as --mixed.

Lesson: git reset: soft, mixed and hard

Discard all local work Exam favourite

```
git reset --hard HEAD
```

Tracked files go back to the last commit. Untracked files stay. Uncommitted edits are lost.

Lesson: git reset: soft, mixed and hard

Fix the last commit

```
git commit --amend
```

Replaces the last commit with a new one: new message and anything newly staged.

Lesson: git reset: soft, mixed and hard

Undo a pushed commit Exam favourite

```
git revert <commit>
```

Creates "Revert ..." as a new commit. --no-edit keeps the default message.

Lesson: git revert: undo in public

Undo a merge commit

```
git revert -m 1 <merge>
```

-m 1 says parent 1 (the main line) is the side to keep.

Lesson: git revert: undo in public

TABLES

Table: what you want to undo · command · safe?

What you want to undo	Command	Safe?
Edits in a file you have not staged	`git restore app.py`	The edits are gone for good
A file you staged by mistake	`git restore --staged app.py`	Yes, the edits stay in the file
The message or content of your last commit (not pushed)	`git commit --amend`	Yes, rewrites one local commit
Your last commits (not pushed)	`git reset <commit>`	Yes with --soft or --mixed, careful with --hard
A commit already pushed and shared	`git revert <commit>`	Yes, adds a new commit, rewrites nothing
Something you reset or deleted by mistake	`git reflog` then reset or branch	Usually rescues it

Lesson: Undo at each of the three levels

Table: mode · moves the branch · resets the index · resets your files

Mode	Moves the branch	Resets the index	Resets your files	Use it to
--soft	yes	no	no	redo the last commits as one: changes stay staged
--mixed (default)	yes	yes	no	unstage and recommit differently: changes stay in your files
--hard	yes	yes	yes	throw everything away and match the commit exactly

Lesson: git reset: soft, mixed and hard

TIPS AND TRAPS

COMMON MISTAKE git restore app.py without --staged overwrites your file from the index. Uncommitted edits it removes are not in any commit, so the reflog cannot bring them back.

Lesson: Undo at each of the three levels

COMMON MISTAKE git reset --hard with uncommitted changes is the most common way people lose work in Git. Run git status and git stash first if in doubt.

Lesson: git reset: soft, mixed and hard

3.2 Stash, tags, .gitignore and .gitattributes

Stash, clean, tags and semantic versions, .gitignore patterns and traps, line endings and Git LFS.

KEY IDEAS

- **Stash** git stash (-u for untracked), stash list, pop (apply and drop), apply (keep), stash branch. A WIP commit is often safer.
- **Tags** Lightweight = plain ref, annotated (-a -m) = tag object for releases. Push with git push origin or --tags. Never move a shared tag.
- **Semver** MAJOR.MINOR.PATCH: breaking, feature, fix. Lower parts reset to 0.
- **.gitignore** Last match wins, ! re-includes, leading / anchors, trailing / means folder, no re-include inside an ignored folder. check-ignore -v explains.
- **Tracked stays tracked** Ignore rules do not untrack files: git rm --cached. A committed secret is leaked: rotate it.

COMMANDS

Save and clean Exam favourite

`git stash` or `git stash push -m "note"`

Saves staged and unstaged changes to tracked files. -u also saves untracked files.

Lesson: git stash: put work aside for a moment

Bring back Exam favourite

`git stash pop`, `git stash apply`

pop applies the newest stash and drops it. apply keeps it on the stack.

Lesson: git stash: put work aside for a moment

Look and tidy

`git stash list`, `git stash show -p stash@{1}`, `git stash drop`

stash@{0} is the newest.

Lesson: git stash: put work aside for a moment

Turn a stash into a branch

`git stash branch <name>`

Creates a branch at the commit where you stashed and applies the stash there, avoiding conflicts.

Lesson: git stash: put work aside for a moment

Push tags Exam favourite

`git push origin v1.0.0`, `git push --tags`

git push alone does not send tags. --follow-tags sends annotated tags that point at pushed commits.

Lesson: Tags and version numbers

List and inspect

`git tag -l "v1.*"`, `git show v1.0.0`, `git describe`

git describe names HEAD by the nearest tag, for example v1.0.0-3-g7c9e1d0 (3 commits after the tag).

Lesson: Tags and version numbers

Semantic versioning Exam favourite

MAJOR.MINOR.PATCH

MAJOR for breaking changes, MINOR for new backwards compatible features, PATCH for backwards compatible bug fixes.

Lesson: Tags and version numbers

Why is this file ignored? Exam favourite

`git check-ignore -v <path>`

Prints the .gitignore file, line number and pattern that matched.

Lesson: .gitignore: keep junk out

Already committed by mistake Exam favourite

`git rm --cached <file>` then commit

.gitignore only affects untracked files. A tracked file stays tracked until you remove it from the index.

Lesson: .gitignore: keep junk out

TABLES

Table: kind · command · stores · use for

Kind	Command	Stores	Use for
Lightweight	`git tag v1.0.0`	Only a ref pointing at the commit	private or temporary marks
Annotated	`git tag -a v1.0.0 -m "Release 1.0.0"`	A tag object with tagger, date, message (and optional signature)	releases

Lesson: Tags and version numbers

Table: pattern · matches

Pattern	Matches
`*.log`	any .log file in any folder
`/todo.txt`	only todo.txt next to the .gitignore
`build/`	any folder called build, and everything inside
`docs/**/*.pdf`	PDFs anywhere under docs
`!keep.log`	brings keep.log back, unless its parent folder is ignored

Lesson: .gitignore: keep junk out

TIPS AND TRAPS

REMEMBER A stash is easy to forget. If it holds real work, a work in progress commit on a branch is often safer and shows up in git log.

Lesson: git stash: put work aside for a moment

COMMON MISTAKE Moving a tag that others have fetched is like changing a published book's edition number. Do not do it. Make a new tag instead.

Lesson: Tags and version numbers

COMMON MISTAKE If a secret was ever committed and pushed, removing it in a new commit is not enough: it is still in history. Treat it as leaked and rotate the password or key at once.

Lesson: .gitignore: keep junk out

STEP 4 Remotes and teamwork

4.1 Remotes: clone, fetch, pull and push

Remotes and origin/main, fetch, pull and push, ahead and behind, upstreams, safe force pushes, prune and refspecs.

KEY IDEAS

- **Remote basics** clone creates origin and origin/* remote-tracking branches. origin/main is your last known copy of the server branch, updated only by fetch, pull and push.
- **Fetch, pull, push** fetch downloads and changes nothing else. pull = fetch + merge or rebase (pull.rebase, --ff-only). push is accepted only as a fast forward.

- **Ahead and behind** ahead = commits only you have, behind = commits only the server has. Both above 0 means diverged.
- **Upstream** git push -u origin sets tracking. git branch -vv shows it with ahead and behind counts.
- **Force safely** Never force push shared branches. On your own branch use --force-with-lease (and --force-if-includes). fetch --prune removes stale origin/* names.

COMMANDS

Clone

Exam favourite

```
git clone <url> [folder]
```

Creates origin, copies all branches as origin/*, checks out the default branch and sets it to track origin.

Lesson: Remotes and git clone

Manage remotes

```
git remote -v, git remote add upstream <url>, git remote set-url origin <url>
```

A repository can have many remotes. upstream is the usual name for the original project when you work on a fork.

Lesson: Remotes and git clone

Fast clones for CI

```
git clone --depth 1, git clone --filter=blob:none
```

--depth 1 is a shallow clone with only the newest commit. --filter=blob:none is a partial clone that fetches file contents only when needed.

Lesson: Remotes and git clone

Set upstream

Exam favourite

```
git push -u origin <branch>
```

After this, plain git push and git pull know where to go. -u is short for --set-upstream.

Lesson: Upstream branches, force pushes and pruning

Safer force push

Exam favourite

```
git push --force-with-lease
```

Overwrites the server branch only if it still points where your origin/branch says. Otherwise it is rejected with stale info.

Lesson: Upstream branches, force pushes and pruning

Delete a remote branch

```
git push origin --delete feature/login
```

Removes the branch on the server, for example after the pull request merged.

Lesson: Upstream branches, force pushes and pruning

Clean up old remote-tracking branches

```
git fetch --prune
```

Removes origin/* names whose branch was deleted on the server. git config --global fetch.prune true does it on every fetch.

Lesson: Upstream branches, force pushes and pruning

TABLES

Table: name · what it is · who moves it

Name	What it is	Who moves it
`main`	your local branch	you, when you commit, merge or reset
`origin/main`	a **remote-tracking branch**: your last known position of main on origin	only git fetch, pull and push
`origin`	a short name for the server URL, stored in .git/config	git remote add, rename, set-url, remove

Lesson: Remotes and git clone

Table: command · what happens to your branch · good when

Command	What happens to your branch	Good when
`git pull --no-rebase`	fetch, then a merge commit joins the two lines	you want history to show exactly what happened
`git pull --rebase`	fetch, then your local commits are replayed on top of origin/main	you want a straight line and your commits are not pushed yet
`git pull --ff-only`	fetch, then move only if no one diverged, otherwise stop	a branch you never commit to directly, such as main on a deploy server

Lesson: fetch, pull and push, ahead and behind

TIPS AND TRAPS

COMMON MISTAKE origin/main does not update by itself. It is a snapshot from your last fetch. If git status says "up to date", it means up to date with that snapshot, not with the server right now.

Lesson: Remotes and git clone

REMEMBER Pick one default once: git config --global pull.rebase true is a popular choice for feature work. Then git pull followed by git push is the everyday rhythm.

Lesson: fetch, pull and push, ahead and behind

COMMON MISTAKE The lease compares with origin/branch. If an editor or a script runs git fetch in the background, origin/branch is already updated and the lease passes without you ever seeing Ben's work. Git 2.30 added --force-if-includes, used together with --force-with-lease, to also check that you had integrated what you fetched.

Lesson: Upstream branches, force pushes and pruning

4.2 Team workflows, pull requests and branch protection

Branching workflows, PRs and MRs, merge methods, branch protection, CODEOWNERS and Conventional Commits.

KEY IDEAS

- **Workflows** GitHub flow: main plus short branches. Gitflow: main plus develop, release and hotfix branches. Trunk based: merge small changes daily behind feature flags. Forking: push to your fork, PR upstream.
- **PR and MR** Server pages that show the diff, run CI and collect reviews before a merge. Keep them small.
- **Merge methods** Merge commit keeps the branch, squash gives one commit, rebase or fast forward gives a straight line.
- **Protect main** Require PRs, approvals and passing status checks (GitHub) or Pipelines must succeed (GitLab). Block force pushes and deletes.
- **CODEOWNERS and messages** CODEOWNERS: last match wins, owners auto requested. Conventional Commits: fix PATCH, feat MINOR, ! or BREAKING CHANGE MAJOR.

TABLES

Table: workflow · long lived branches · best for · main risk

Workflow	Long lived branches	Best for	Main risk
GitHub flow	main only	web apps that deploy often	needs good CI, because main must always work
Gitflow	main and develop	products with numbered, scheduled releases and several supported versions	many branches and slow, painful merges
Trunk based	main (the trunk)	teams with strong automated tests who merge at least daily	unfinished work must hide behind feature flags
Forking	your fork plus the original (upstream)	open source, where outsiders cannot push	keeping the fork in sync

Lesson: Four branching workflows

Table: github merge button · gitlab merge method · resulting history

GitHub merge button	GitLab merge method	Resulting history
Create a merge commit	Merge commit	all branch commits plus a merge commit, the same as git merge --no-ff
Squash and merge	Squash commits option (with the merge commit methods a merge commit is added too)	one new commit holding the whole change
Rebase and merge	Fast-forward merge	the branch commits replayed on top, straight line, no merge commit
(no equivalent)	Merge commit with semi-linear history	merge commit, but only allowed when the branch is already rebased on the target

Lesson: Pull requests and merge requests

Table: rule · github (settings, branches or rules) · gitlab (settings, repository, protected branches)

Rule	GitHub (Settings, Branches or Rules)	GitLab (Settings, Repository, Protected branches)
No direct pushes	Require a pull request before merging	Allowed to push and merge: No one
Reviews	Required approvals, require review from Code Owners	Approval rules, Require approval from code owners (Premium)
Tests must pass	Require status checks to pass, pick the CI jobs	Settings, Merge requests, Merge checks: Pipelines must succeed
No history rewrites	Force pushes are blocked by default in a rule	Allowed to force push: off
No deleting the branch	Deletions are blocked by default in a rule	A protected branch cannot be deleted with git push, only by a Maintainer in the UI or API
Straight history	Require linear history	Merge method: Fast-forward merge

Lesson: Protect main: branch rules and code owners

Table: message · meaning · semver effect

Message	Meaning	Semver effect
`fix(calc): handle divide by zero`	a bug fix	PATCH
`feat(calc): add percent()`	a new feature	MINOR
`feat!: drop Python 3.9` or a footer `BREAKING CHANGE: ...`	a breaking change	MAJOR
`docs:`, `test:`, `ci:`, `refactor:`, `chore:`	allowed extra types	none by default

Lesson: Good commit messages and Conventional Commits

TIPS AND TRAPS

IN THE EXAM Gitflow was published by Vincent Driessen in 2010. In 2020 he added a note to the article saying that for software delivered continuously, such as web apps, a simpler workflow like GitHub flow is usually a better fit. Gitflow still suits software with explicit versions.

Lesson: Four branching workflows

COMMON MISTAKE Squash and merge makes main easy to read, but the branch commits do not appear on main. If someone keeps working on the old branch afterwards, Git does not know those commits were merged and conflicts follow. Start a fresh branch after a squash merge.

Lesson: Pull requests and merge requests

COMMON MISTAKE If your branch rule requires a status check called unit-tests, the job name in the workflow must stay unit-tests. Renaming the job makes the required check wait forever. GitHub also advises keeping job names unique across workflows for this reason.

Lesson: Protect main: branch rules and code owners

4.3 Power tools: bisect, worktree, submodules, hooks and signing

bisect, worktree, submodules, hooks, grep, sparse checkout, signed commits and archive.

KEY IDEAS

- **git bisect** start, bad, good , then good or bad or skip until the first bad commit, then reset. bisect run script: 0 good, 125 skip, 1 to 127 bad. About log2(n) tests.
- **worktree** git worktree add gives another folder on another branch, sharing one .git. One branch per worktree.
- **Submodules** Pinned commit of another repository. .gitmodules holds the URL. Clone with --recurse-submodules or run submodule update --init --recursive. CI needs submodules: recursive or GIT_SUBMODULE_STRATEGY.
- **Hooks** pre-commit, commit-msg, pre-push on the client (skippable with --no-verify, not cloned), pre-receive and post-receive on the server. Share with core.hooksPath or the pre-commit framework, and repeat checks in CI.
- **More tools** git grep searches tracked files, sparse-checkout limits folders, commit -S and tag -s sign (SSH or GPG), git archive exports a tag without .git.

COMMANDS

Add and clone Exam favourite

```
git submodule add <url> <path>, git clone --recurse-submodules <url>
```

A plain clone leaves submodule folders empty.

Lesson: worktree and submodules

After a pull Exam favourite

```
git submodule update --init --recursive
```

Checks out the commit each submodule is pinned to.

Lesson: worktree and submodules

In CI

`actions/checkout` with `submodules: recursive`, GitLab variable `GIT_SUBMODULE_STRATEGY: recursive`

Both platforms skip submodules unless asked.

Lesson: worktree and submodules

Search tracked files Exam favourite

```
git grep -n "def add"
```

Faster than grep on big repositories and ignores untracked junk. Add a commit or tag to search an old version: git grep -n TODO v1.0.

Lesson: grep, sparse checkout, signing and archive

Work on part of a monorepo

```
git sparse-checkout set app tests
```

Only these folders appear in the working tree. git sparse-checkout disable brings everything back.

Lesson: grep, sparse checkout, signing and archive

Sign a commit or tag

Exam favourite

```
git commit -S, git tag -s v1.0.0
```

Proves who made it. Set commit.gpgsign true to sign every commit.

Lesson: grep, sparse checkout, signing and archive

Export a release

```
git archive --format=zip -o calc-app.zip v1.0.0
```

Writes the files of a tag or commit to a zip or tar, without the .git folder.

Lesson: grep, sparse checkout, signing and archive

TABLES

Table: exit code of the script · meaning for bisect run

Exit code of the script	Meaning for bisect run
0	good
1 to 127, except 125	bad
125	cannot test this commit, skip it
above 127	stop bisecting

Lesson: git bisect: find the commit that broke it

Table: hook · runs · typical use

Hook	Runs	Typical use
pre-commit	before the commit message is asked for	format code, lint, find secrets
commit-msg	after the message is written	check Conventional Commits format
pre-push	before objects are sent	run fast unit tests
pre-receive and update	on the server, when a push arrives	reject pushes that break rules (GitLab server hooks, GitHub uses rulesets instead)
post-receive	on the server, after the push	notify, start deployments

Lesson: Hooks: scripts Git runs for you

TIPS AND TRAPS

REMEMBER — Bisect works best when every commit builds and passes tests on its own. Huge commits that mix many changes also make the answer less useful. This is one more reason for small, atomic commits.

Lesson: git bisect: find the commit that broke it

REMEMBER — A branch can be checked out in only one worktree at a time. All worktrees share the same .git objects, so a commit made in one is visible in the others at once.

Lesson: worktree and submodules

COMMON MISTAKE — Files in .git/hooks are not committed and not cloned, and anyone can skip them with --no-verify. Use hooks for fast feedback on your laptop, and run the same checks again in CI, which no one can skip.

Lesson: Hooks: scripts Git runs for you

STEP 5 Git internals

5.1 Inside .git: blobs, trees, commits and tags

The four object types, how ids are computed and stored, tree modes, Merkle integrity and the move to SHA-256.

KEY IDEAS

- **Four objects** blob (content), tree (names, modes, ids), commit (tree, parents, author, committer, message), annotated tag (object, type, name, tagger, message).
- **Object id** SHA-1 of the type, a space, the size, a zero byte and the content. 40 hex digits (SHA-256: 64). Stored zlib compressed at .git/objects/xx/rest.
- **Plumbing to inspect** git hash-object [-w], git cat-file -t, -s, -p, git rev-parse. Tree modes: 100644 file, 100755 executable, 120000 symlink, 040000 tree, 160000 submodule.
- **Merkle chain** Commit ids depend on every file and every parent, so changing history changes every later id. Identical content is stored once.
- **SHA-1 and SHA-256** SHAttered 2017, SHA-1DC since Git 2.13, --object-format=sha256 since 2.29, planned default in Git 3.0.

COMMANDS

Hash without storing Exam favourite

```
git hash-object <file>
```

Add -w to also write the object into .git/objects.

Lesson: How an object gets its name

Read any object Exam favourite

```
git cat-file -t <id>, -s, -p
```

-t prints the type, -s the size, -p the content in readable form.

Lesson: How an object gets its name

Where it is stored

```
.git/objects/ce/013625...
```

First 2 hex digits are the folder, the other 38 the file name. The file is zlib compressed.

Lesson: How an object gets its name

Short ids

```
git rev-parse --short HEAD
```

Any unique prefix works, usually 7 or more digits.

Lesson: How an object gets its name

TABLES

Table: object · holds · points to

Object	Holds	Points to
blob	the bytes of one file, with no name and no permissions	nothing
tree	one folder: a list of names, each with a mode and an object id	blobs and other trees
commit	one tree id, parent ids, author, committer, dates and the message	one tree and zero or more parent commits
tag (annotated)	object id, type, tag name, tagger, date and message	usually a commit

Lesson: Four kinds of objects

Table: mode in a tree · means

Mode in a tree	Means
100644	normal file
100755	executable file
120000	symbolic link
040000	a subfolder (another tree)
160000	a submodule commit (gitlink)

Lesson: Four kinds of objects

TIPS AND TRAPS

REMEMBER A blob has no file name. The name lives in the tree. That is why renaming a file creates a new tree but reuses the same blob, and why Git detects renames by comparing contents.

Lesson: Four kinds of objects

IN THE EXAM The id depends only on the content, never on the file name, date or computer. Ten copies of the same file in a repository are stored once.

Lesson: How an object gets its name

COMMON MISTAKE SHA-1 and SHA-256 repositories cannot simply push to each other yet, and some hosting services do not accept SHA-256 repositories. Check your server before you switch.

Lesson: Why history cannot be changed quietly

5.2 Refs, the index, plumbing and packfiles

Refs and HEAD, plumbing behind add and commit, index stages, packfiles, deltas, gc and fsck.

KEY IDEAS

- **Refs** Names for ids: refs/heads (branches), refs/tags, refs/remotes, packed-refs. HEAD is a symbolic ref to a branch. Change refs with update-ref, never by hand. reftable comes in Git 2.45 and is planned as default in 3.0.
- **Plumbing behind add and commit** hash-object -w, update-index --add, write-tree, commit-tree, update-ref. Porcelain is for people, plumbing is stable for scripts.
- **Index stages** 0 normal, 1 base, 2 ours, 3 theirs during a conflict. git ls-files --stage shows them.
- **Packs and gc** Loose objects are packed by gc, similar objects stored as deltas. gc prunes unreachable objects after two weeks, reflog entries keep commits alive for 30 or 90 days. fsck checks and lists dangling objects.
- **CI clones** actions/checkout fetches one commit by default (fetch-depth: 1). Use fetch-depth: 0 when a job needs tags or history. GitLab uses GIT_DEPTH.

COMMANDS

Pack and clean Exam favourite

```
git gc
```

Packs loose objects and refs, and removes unreachable objects older than the grace period (two weeks by default, gc.pruneExpire).

Lesson: Packfiles, deltas and garbage collection

Check the database Exam favourite

```
git fsck
```

Verifies every object and reports dangling objects that nothing points to.

Lesson: Packfiles, deltas and garbage collection

Count

```
git count-objects -vH
```

Loose and packed objects and their sizes.

Lesson: Packfiles, deltas and garbage collection

Background upkeep

```
git maintenance start
```

Schedules prefetch, commit-graph and repacking in the background for big repositories.

Lesson: Packfiles, deltas and garbage collection

TABLES

Table: file in .git · holds · changed by

File in .git	Holds	Changed by
`HEAD`	`ref: refs/heads/main` (or an id when detached)	git switch, git checkout
`refs/heads/main`	the id of the newest commit on main	commit, merge, reset, pull
`refs/tags/v1.0`	a commit id or a tag object id	git tag
`refs/remotes/origin/main`	the last fetched id of main on origin	fetch, pull, push
`packed-refs`	many refs in one file, after git pack-refs or git gc	gc, pack-refs
`logs/HEAD`, `logs/refs/...`	the reflog: every old value of each ref	every ref update

Lesson: Refs and HEAD are small text files

Table: porcelain · plumbing steps

Porcelain	Plumbing steps
`git add f`	hash-object -w f, then update-index --add f
`git commit -m m`	write-tree, commit-tree <tree> -p HEAD -m m, update-ref refs/heads/<branch> <new id>
`git branch b`	update-ref refs/heads/b HEAD
`git switch b`	read-tree and checkout-index for the files, symbolic-ref HEAD refs/heads/b

Lesson: add and commit done by hand

Table: path · what it is

Path	What it is
`HEAD`	the current branch
`config`	local settings, remotes, branch upstreams
`index`	the staging area
`objects/`	loose objects in xx/ folders, packs in pack/
`refs/`, `packed-refs`	branches, tags, remote-tracking branches, stash
`logs/`	reflogs
`hooks/`	sample and real hook scripts
`info/exclude`	personal ignore rules
`MERGE_HEAD`, `ORIG_HEAD`, `FETCH_HEAD`	temporary refs during a merge, after a reset or rebase, and after a fetch

Lesson: The whole .git folder

TIPS AND TRAPS

COMMON MISTAKE Never edit ref files with a text editor. Use git update-ref, which writes safely, checks the old value if you ask and records the change in the reflog.

Lesson: Refs and HEAD are small text files

IN THE EXAM Plumbing output is stable and meant for scripts. Porcelain output such as git status can change between versions, so scripts should use plumbing or the --porcelain options.

Lesson: add and commit done by hand

COMMON MISTAKE Commits you can still see in the reflog are reachable from it, so gc keeps them until the reflog entry expires (90 days by default, 30 for entries that are not reachable from the branch). That is the safety net from chapter 3.1. --prune=now on a repository you care about removes that net for loose objects nothing points to.

Lesson: Packfiles, deltas and garbage collection

STEP 6 CI/CD foundations

6.1 CI/CD foundations: pipelines, runners, artifacts and roles

CI, CD and pipelines, jobs, stages, needs and runners, YAML, artifacts versus cache, secrets, Dev and QA roles.

KEY IDEAS

- **CI and CD** CI: merge small changes often, every push built and tested. Continuous delivery: always releasable, a person releases. Continuous deployment: a green pipeline releases by itself.
- **Pipeline parts** Pipeline or workflow run, jobs on fresh runners, steps or script lines. Stages wait for the whole previous stage, needs wait only for named jobs. Time = longest path.
- **YAML** Spaces not tabs, key: value, - list items, run: | for blocks. Quote versions ("3.10").
- **Artifacts versus cache** Artifacts are results to keep and pass on (build packages, reports). Cache is a speed up for dependencies, safe to lose.
- **Secure and promote** Secrets outside YAML, least permissions, pin third party actions by SHA (tj-actions 2025, Codecov 2021). Build once, promote the same artifact through dev, QA, staging and production (Knight Capital 2012).

COMMANDS

Variables

GitHub: `env:` and `vars.NAME`. GitLab: `variables:` and project CI/CD variables

Plain settings such as a test URL. Visible in logs if printed.

Lesson: Artifacts, cache, variables and secrets

Secrets Exam favourite

GitHub: `${{ secrets.NAME }}`. GitLab: masked and protected CI/CD variables

Stored encrypted and hidden in logs. Never write a secret into the YAML file.

Lesson: Artifacts, cache, variables and secrets

Least privilege Exam favourite

GitHub: `permissions: contents: read`. GitLab: protected variables only on protected branches

Give each job only the access it needs.

Lesson: Artifacts, cache, variables and secrets

Pin third party actions Exam favourite

`uses: owner/action@<full commit SHA>`

A tag such as @v4 can be moved to new code. A full commit id cannot.

Lesson: Artifacts, cache, variables and secrets

TABLES

Table: term · github actions · gitlab ci/cd

Term	GitHub Actions	GitLab CI/CD
the config file	`.github/workflows/*.yml` (many files allowed)	`.gitlab-ci.yml` at the root (one file, can include others)
one run	workflow run	pipeline
a unit of work on one machine	job	job
the commands inside a job	steps (run or uses)	script lines
the machine	runner (GitHub hosted or self hosted)	runner (instance runners on GitLab.com or your own)
grouping	needs between jobs	stages, plus needs to skip waiting

Lesson: What CI and CD mean, and why teams need them

Table: yaml · means

YAML	Means
`image: python:3.13-slim`	a key with a string value
`branches: [main, dev]`	a list written on one line
`steps:` then lines starting with `-`	a list written as lines
`run: |`	a block of text: every following indented line is kept, newlines included
`# note`	a comment

Lesson: Inside a pipeline: jobs, stages, runners and YAML

Table: · artifact · cache

	Artifact	Cache
What for	results you need later: a build package, test reports, coverage	things you can rebuild but would rather not: downloaded dependencies
Who uses it	later jobs in the same run, people, other pipelines	later runs of the same or another job
If it is missing	the pipeline is wrong	the job is only slower
GitHub Actions	actions/upload-artifact and download-artifact, kept 90 days by default	actions/cache or the cache option of setup actions, 10 GB per repository
GitLab CI/CD	artifacts: paths and expire_in (30 days on GitLab.com by default)	cache: key and paths

Lesson: Artifacts, cache, variables and secrets

TIPS AND TRAPS

COMMON MISTAKE YAML guesses types. python-version: 3.10 is read as the number 3.1, and the job installs the wrong Python. Always quote versions: python-version: "3.10". Old YAML 1.1 parsers also read a bare no or off as false.

Lesson: Inside a pipeline: jobs, stages, runners and YAML

IN THE EXAM If each environment rebuilds from source, QA tested a different file from the one users get. Build once, store it as an artifact or in a package registry, and promote that same file.

Lesson: Dev and QA in a pipeline, and build once, promote many

STEP 7 GitHub Actions

7.1 GitHub Actions: events, jobs, matrix and contexts

Workflow anatomy, events and filters, jobs, needs, matrix, contexts, outputs, permissions and safety.

KEY IDEAS

- **Workflow file** .github/workflows/*.yml with on, jobs, runs-on, steps (uses or run). checkout first. Quote versions.
- **Events** push and pull_request with branches, tags, paths filters, workflow_dispatch with inputs, schedule cron in UTC on the default branch. * stops at /, ** does not.
- **Jobs** Parallel by default, needs for order, if for conditions, matrix for combinations (max 256, fail-fast default true). Outputs through $GITHUB_OUTPUT.
- **Contexts** github, env, vars, secrets, matrix, steps, needs, inputs, runner inside ${{ }}.
- **Safety** permissions least privilege, concurrency with cancel-in-progress, environments with reviewers, untrusted text through env only, fork PRs get no secrets, job timeout default 360 minutes.

COMMANDS

Branch and tag filters Exam favourite

```
branches: [main, 'release/**'], tags: ['v*']
```

* matches any characters except /, ** also matches /.

Lesson: Events and filters: when does it run

Path filters

```
paths: ['app/**'], paths-ignore: ['docs/**', '**.md']
```

Skip the run when only docs changed. Use paths or paths-ignore for one event, not both.

Lesson: Events and filters: when does it run

Cron Exam favourite

```
- cron: '30 2 * * 1-5'
```

Minute, hour, day of month, month, day of week. This is 02:30 UTC Monday to Friday.

Lesson: Events and filters: when does it run

Least privilege token Exam favourite

```
permissions: contents: read
```

Every run gets a GITHUB_TOKEN. Set only what the jobs need. Add actions: read to download artifacts from other runs, packages: write to publish packages.

Lesson: Permissions, concurrency and safe workflows

One run per branch Exam favourite

```
concurrency: group: ci-${{ github.ref }} cancel-in-progress: true
```

A new push cancels the older run on the same branch, saving runner minutes.

Lesson: Permissions, concurrency and safe workflows

Environments

`environment: qa` on a job

An environment can hold its own secrets and protection rules such as required reviewers before the job starts.

Lesson: Permissions, concurrency and safe workflows

Debug logs

Re-run jobs with debug logging, or secret `ACTIONS_STEP_DEBUG` set to true

Shows extra output from actions.

Lesson: Permissions, concurrency and safe workflows

TABLES

Table: key · level · meaning

Key	Level	Meaning
`on`	workflow	events and filters
`permissions`	workflow or job	what the automatic GITHUB_TOKEN may do
`env`	workflow, job or step	environment variables
`concurrency`	workflow or job	at most one run per group, optionally cancel the older one
`runs-on`	job	which runner
`needs`, `if`	job	order and conditions
`strategy.matrix`	job	many jobs from one definition
`timeout-minutes`	job or step	cancel after this many minutes. The job default is 360
`uses` or `run`, `with`, `id`	step	an action or shell commands, inputs, a name to read outputs

Lesson: A workflow file, line by line

Table: event · starts when · good for

Event	Starts when	Good for
`push`	commits or tags are pushed	CI on main, release on tags
`pull_request`	a PR is opened, gets new commits (synchronize) or is reopened, by default	checking a change before merge. The run tests the merge result of the PR with its base
`workflow_dispatch`	someone presses Run workflow, or calls the API or gh workflow run	manual QA runs with inputs
`schedule`	a cron time in UTC, on the default branch only	nightly regression
`workflow_run`	another workflow finished	chaining a QA workflow after CI

Lesson: Events and filters: when does it run

Table: context · example · holds

Context	Example	Holds
`github`	`github.sha`, `github.ref_name`, `github.event_name`	facts about the event and run
`env`, `vars`, `secrets`	`secrets.DEPLOY_KEY`	variables and encrypted secrets
`matrix`, `strategy`	`matrix.os`	the current combination
`steps`, `needs`	`needs.build.outputs.version`	outputs of earlier steps and jobs
`inputs`	`inputs.ci_run_id`	workflow_dispatch inputs
`runner`	`runner.os`	Linux, Windows or macOS

Lesson: Jobs, needs, if, matrix and outputs

TIPS AND TRAPS

COMMON MISTAKE A workflow added on a feature branch with on: push: branches: [main] does not run for pushes to that feature branch. Beginners often think CI is broken. Add pull_request, open a PR, or push to main.

Lesson: Events and filters: when does it run

REMEMBER Scheduled runs can start late when GitHub is busy, especially at the top of the hour. Pick an odd minute such as 2:30 rather than 0:00.

Lesson: Events and filters: when does it run

IN THE EXAM A matrix can produce at most 256 jobs per workflow run. Six operating systems times six Python versions times eight databases is already 288.

Lesson: Jobs, needs, if, matrix and outputs

COMMON MISTAKE Pull requests from forks get a read only GITHUB_TOKEN and no repository secrets. Do not switch to pull_request_target to work around this if the job runs code from the pull request, because that event runs with your secrets.

Lesson: Permissions, concurrency and safe workflows

7.2 GitHub Actions from scratch: a green build on every commit

A complete GitHub Actions setup from an empty repository, explained line by line, with Dev and QA changes and a QA regression workflow.

KEY IDEAS

- **From scratch** Test locally, create an empty repo, commit the project with .github/workflows/ci.yml, push to main, watch the Actions tab, download the artifact, break it on a branch, fix it, then require the checks on main.
- **ci.yml** push and pull_request on main plus workflow_dispatch, permissions contents: read, concurrency per ref. lint, unit-tests, build, smoke-tests chained by needs. Checkout in every job.
- **Artifacts in the flow** build uploads calc-app once (if-no-files-found: error), smoke-tests downloads it with download-artifact in the same run, reports kept with if: always().
- **Dev versus QA changes** Dev: triggers, matrix, cache, cancel-in-progress. QA: test the artifact, separate scheduled suites, fail-fast false, longer retention, inputs.
- **QA regression** schedule plus workflow_dispatch input, actions: read, GH_TOKEN, gh run list for the last green run, gh run download -n calc-app, inputs passed through env.

TABLES

Table: job · needs · makes · fails when

Job	Needs	Makes	Fails when
lint	nothing	nothing	a .py file has a syntax error
unit-tests	lint	unit-report (14 days)	a unit test fails
build	unit-tests	calc-app (30 days)	the zip cannot be made
smoke-tests	build	smoke-report (14 days)	the packaged app is broken or not the packaged one

Lesson: The CI workflow, line by line

Table: you see · cause · fix

You see	Cause	Fix
No run starts after a push	the file is not in .github/workflows/, the YAML is invalid, or the branch filter does not match	check the path, open the file on GitHub (it shows YAML errors), check on: push: branches
No such file or directory: requirements.txt	no actions/checkout step in that job	add uses: actions/checkout as the first step of every job
ModuleNotFoundError: No module named 'app'	pytest started without the project on the import path	run python -m pytest, or set PYTHONPATH
download-artifact cannot find calc-app	the name differs from the upload, or the job lacks needs: build	use the same name and add needs
gh run download fails with HTTP 403 or 404	the workflow token lacks actions: read, or the artifact expired	add the permission, check the retention period
A required check stays at Expected and never reports	the rule names a job that no longer exists	rename the check in the rule or keep the job id

Lesson: The QA regression workflow and common errors

TIPS AND TRAPS

REMEMBER Change one thing at a time and push it on a branch with a pull request. The pull request runs the workflow file from the branch, so you see the result before main is affected.

Lesson: What Dev and QA change, and why

7.3 GitHub Actions artifacts and caching: publish, pull and speed up

Publishing and pulling artifacts five ways, retention and immutability, caching with keys and restore-keys, and the limits.

KEY IDEAS

- **upload-artifact** name (unique per run), path with ! excludes, if-no-files-found, retention-days (default 90), outputs artifact-id, artifact-url, artifact-digest. Immutable since v4, overwrite: true replaces. Hidden files skipped, permissions reset.
- **Pull artifacts** Same run: download-artifact name or pattern plus needs. Other run: github-token, repository, run-id. People: run page. Terminal: gh run download -n name -D dir. API: /repos/{owner}/{repo}/actions/artifacts/{id}/zip.
- **actions/cache** key from runner.os and hashFiles, restore-keys prefixes, cache-hit only on exact match, saved only on a miss. Branch scope plus default branch. 10 GB per repo, 7 days unused.
- **Choose** Artifact for results of a run, cache for speed, Release or Packages for versions users install.
- **Safety** No secrets in caches or artifacts, read only cache for fork triggers, record digests, use attestations for release builds.

TABLES

Table: fact · detail

Fact	Detail
Immutable	Since version 4 an artifact cannot be changed after upload. Uploading the same name twice in one run fails, unless you set overwrite: true, which deletes the old one and gives a new id.
Zipped	By default the files are zipped. archive: false uploads one file as it is.
Hidden files	Files and folders starting with a dot are skipped unless include-hidden-files: true, so .env files are not leaked by accident.
Permissions	Zipped artifacts lose file permissions: files become 644, so an executable bit is lost. Put a tar file inside if you need it.
Limits	At most 500 artifacts per job. Storage counts against the account quota.

Lesson: Publishing artifacts: upload-artifact

Table: who · from where · how

Who	From where	How
A later job	the same run	`actions/download-artifact` with `name`, plus `needs` on the uploading job
A job in another workflow	another run, same repository	download-artifact with `github-token`, `run-id` (and `repository`), or `gh run download`
A person	any run	the Artifacts section at the bottom of the run summary page
A person or script with a terminal	any run	`gh run download <run-id> -n calc-app -D dir`
Any tool	any run	the REST API: list, then download the zip

Lesson: Pulling artifacts: five ways

Table: rule · detail

Rule	Detail
Scope	A run can restore caches from its own branch and from the default branch. A pull request can also use its base branch. Branches do not see each other's caches.
Save	A cache is saved at the end of the job only when the key had no exact hit. An existing key is never overwritten.
Limits	10 GB per repository by default. Entries not used for 7 days are removed, and the oldest go first when the limit is reached.
Output	cache-hit is true only for an exact key match.
Security	Workflows from low trust triggers such as fork pull requests get read only access to the default branch cache, to prevent cache poisoning.

Lesson: Caching dependencies: actions/cache

Table: need · use · kept

Need	Use	Kept
Pass the build from build to test jobs	artifact	retention-days, up to 90 by default
Keep test reports, screenshots, logs	artifact with if: always()	shorter retention is fine
Speed up installs	cache	until unused for 7 days or evicted
Ship a version to users	GitHub Release asset	until deleted
Share a library with other projects	GitHub Packages or PyPI and similar registries	until deleted

Lesson: Artifact, cache, release or package

TIPS AND TRAPS

COMMON MISTAKE A matrix job that uploads with a fixed name fails from the second combination on, because names must be unique. Add the matrix values: name: report-${{ matrix.os }}-${{ matrix.python }}.

Lesson: Publishing artifacts: upload-artifact

IN THE EXAM Artifacts are for files of one run, kept for a limited time. For a release that users install, publish it to GitHub Releases or GitHub Packages, which keep it until you delete it.

Lesson: Pulling artifacts: five ways

COMMON MISTAKE Never cache secrets or build outputs that must be correct. Caches can be restored from a partial match, so they may hold stale files. If a job must use exactly these files, it needs an artifact, not a cache.

Lesson: Caching dependencies: actions/cache

STEP 8 GitLab CI/CD

8.1 GitLab CI/CD: stages, rules, needs and runners

GitLab pipeline structure, rules and workflow, pipeline types, needs, parallel, extends, include, runners and variables.

KEY IDEAS

- **File and stages** .gitlab-ci.yml at the root. Default stages .pre, build, test, deploy, .post. Same stage in parallel, next stage after the previous one succeeds. script is required.
- **Rules** Top down, first match decides, when never excludes, no match excludes. workflow:rules decide the whole pipeline and stop duplicate branch plus MR pipelines.
- **Pipeline types** Branch (CI_COMMIT_BRANCH), tag (CI_COMMIT_TAG), merge request (CI_PIPELINE_SOURCE merge_request_event), schedule, web.
- **Speed and reuse** needs (up to 50, needs: [] starts at once), parallel N or matrix (up to 200), hidden .jobs with extends, include for shared YAML.
- **Runners and variables** Instance, group, project runners. Docker, shell, Kubernetes executors. tags select runners. Masked and protected variables for secrets.

COMMANDS

Skip waiting for the stage — Exam favourite

`needs: [build]`

The job starts as soon as build is done, even if other jobs of the build stage still run. needs: [] starts at once. Up to 50 jobs in one needs list.

Lesson: needs, parallel, extends and include

Run a job many times — Exam favourite

`parallel: 3` or `parallel: matrix:`

parallel: 3 gives jobs 1/3, 2/3, 3/3 with CI_NODE_INDEX and CI_NODE_TOTAL for test splitting. parallel:matrix sets variables, at most 200 combinations.

Lesson: needs, parallel, extends and include

Reuse a job template

`.python-job:` then `extends: .python-job`

Jobs whose name starts with a dot are hidden and never run, so they work as templates.

Lesson: needs, parallel, extends and include

Split the file

`include: local:, project:, remote:, template:, component:`

Share one pipeline definition between many projects.

Lesson: needs, parallel, extends and include

Project variables Exam favourite

Settings, CI/CD, Variables

Masked hides the value in job logs. Protected passes it only to pipelines on protected branches and tags.

Lesson: Runners, executors and variables

Predefined variables

`$CI_COMMIT_SHA`, `$CI_PROJECT_DIR`, `$CI_JOB_TOKEN`, `$CI_API_V4_URL`

Set by GitLab in every job.

Lesson: Runners, executors and variables

Pick a runner

```
tags: [linux, docker]
```

A job only runs on runners that have all of its tags.

Lesson: Runners, executors and variables

TABLES

Table: keyword · where · does

Keyword	Where	Does
`stages`	top	order of stages
`default`	top	image, before_script, after_script, cache, retry and more for every job
`variables`	top or job	CI/CD variables
`workflow`	top	whether the whole pipeline is created
`include`	top	pull in YAML from other files, projects, templates or components
`stage`, `script`	job	where it runs and the shell commands (script is required)
`image`, `services`	job	Docker image for the job, and side containers such as a database
`rules`, `needs`	job	when the job is added, and which jobs it waits for
`artifacts`, `cache`	job	files to keep and pass on, files to reuse between pipelines

Lesson: One file, stages and jobs

Table: pipeline type · started by · useful variables

Pipeline type	Started by	Useful variables
Branch pipeline	a push to a branch	CI_COMMIT_BRANCH (not set in merge request or tag pipelines)
Tag pipeline	pushing a tag	CI_COMMIT_TAG
Merge request pipeline	creating or updating a merge request, when a job has rules for it	CI_PIPELINE_SOURCE == "merge_request_event", CI_MERGE_REQUEST_TARGET_BRANCH_NAME
Scheduled pipeline	Build, Pipeline schedules	CI_PIPELINE_SOURCE == "schedule"
Manual run	Build, Pipelines, New pipeline	CI_PIPELINE_SOURCE == "web"

Lesson: rules, workflow and pipeline types

Table: runner scope · available to

Runner scope	Available to
Instance	every project on the GitLab instance (GitLab hosted runners on GitLab.com are instance runners)
Group	every project in a group
Project	one project

Lesson: Runners, executors and variables

Table: executor · runs each job · note

Executor	Runs each job	Note
Docker	in a fresh container from the job image	the most common, clean for each job
Shell	directly on the runner machine	simple, but jobs share the machine and its leftovers
Kubernetes	in a pod in a cluster	scales out for many parallel jobs

Lesson: Runners, executors and variables

TIPS AND TRAPS

IN THE EXAM script lines run in order, and the job fails at the first command that exits with a non zero code. after_script still runs, but its result does not change the job status.

Lesson: One file, stages and jobs

COMMON MISTAKE Add a rule for merge requests to one job without workflow:rules and a single push to a branch with an open merge request can start two pipelines, a branch pipeline and a merge request pipeline. The workflow block above is the documented fix.

Lesson: rules, workflow and pipeline types

8.2 GitLab CI/CD from scratch: a green pipeline on every commit

A complete GitLab CI/CD setup from an empty project, line by line, with Dev and QA changes and a GitHub comparison.

KEY IDEAS

- **From scratch** Blank project without README, push the repo with .gitlab-ci.yml, watch Build, Pipelines, check the Tests tab and job artifacts, break and fix in a merge request, then Pipelines must succeed and protected main.
- **The sample file** workflow rules (MR, skip branch with open MR, branches), stages lint, test, build, qa, image python:3.13-slim, cache key files on requirements.txt, junit reports, build artifact with name and expire_in, smoke-tests with needs artifacts, nightly job on schedule.
- **Tools** Pipeline editor (Validate, Visualize), Tests tab, job artifacts Download and Browse, Build, Pipeline schedules.
- **Dev versus QA changes** Dev: needs, interruptible, cache, extends, protected variables. QA: test the artifact, parallel suites, JUnit reports, schedules and web runs, longer expiry for releases.
- **Common errors** No pipeline (YAML or workflow rules), stuck job (runner tags), duplicate pipelines (no workflow rules), missing artifact (needs), cache path outside the project.

TABLES

Table: you see · cause · fix

You see	Cause	Fix
No pipeline after the push	the file name or location is wrong, YAML is invalid, or workflow rules matched nothing (for example a tag push)	open the Pipeline editor, read the error, check the workflow rules
This job is stuck because you do not have any active runners	no runner matches, often because of tags: on the job	remove or fix tags, or enable instance runners under Settings, CI/CD, Runners
Two pipelines for one push	a job has merge request rules but there is no workflow block	add the workflow rules from this chapter
ModuleNotFoundError in smoke-tests	the artifact did not arrive, so dist/calc-app.zip is missing	check needs: build with artifacts: true and that build uploaded the path
Cache never restores	the cache path is outside the project folder	set PIP_CACHE_DIR inside $CI_PROJECT_DIR, as in the sample

Lesson: What Dev and QA change on GitLab, and why

Table: idea · github actions (ci.yml) · gitlab ci/cd (.gitlab-ci.yml)

Idea	GitHub Actions (ci.yml)	GitLab CI/CD (.gitlab-ci.yml)
When to run	`on: push / pull_request` with branch filters	`workflow: rules` with pipeline source and branch variables
Machine	`runs-on: ubuntu-latest`, then setup-python	`image: python:3.13-slim` on an instance runner
Get the code	`actions/checkout` in every job	automatic, the runner clones before the script
Order	`needs` only	`stages`, plus `needs` to skip waiting
Keep the build	`actions/upload-artifact`	`artifacts: paths`
Use the build later	`actions/download-artifact` + needs	automatic for later stages, or `needs: artifacts: true`
Test report in the UI	upload the XML, or a reporting action	`artifacts: reports: junit`, shown in the MR and Tests tab
Cache	`cache: pip` in setup-python	`cache: key: files:` plus a cache folder inside the project
Nightly	separate workflow with `schedule`	a pipeline schedule plus a job rule on `CI_PIPELINE_SOURCE`
Merge only when green	branch rule: required status checks	Merge checks: Pipelines must succeed

Lesson: The same pipeline on GitHub and GitLab

TIPS AND TRAPS

IN THE EXAM

GitLab uploads artifacts of all jobs and passes them on to jobs in later stages automatically. needs and dependencies control which ones a job downloads. dependencies: [] downloads none, which saves time in jobs that do not need files.

Lesson: The .gitlab-ci.yml, line by line

8.3 GitLab artifacts and caching: keep, pass on and pull builds

GitLab artifacts and reports, passing them between jobs, pulling with UI, API, glab and job token, and caching.

KEY IDEAS

- **artifacts keyword** paths, exclude, name, when (on_success, on_failure, always), expire_in (GitLab.com default 30 days), access, reports: junit. Latest successful artifacts per ref are kept beyond expire_in. Keep button. 1 GB on GitLab.com.
- **Between jobs** Earlier stages by default, dependencies: [list] or [] to narrow, needs: [job] to start early and fetch, needs:project for other projects (Premium).
- **Pull artifacts** UI: pipelines, jobs, job page, MR, Build Artifacts. API: /projects/:id/jobs/artifacts/:ref/download?job=name, /jobs/:job_id/artifacts?file_type=junit. CLI: glab job artifact . CI_JOB_TOKEN with JOB-TOKEN header plus allowlist.
- **Cache** cache key files (up to two), prefix, paths inside the project, policy pull-push, pull or push. Best effort only.
- **Releases** Artifacts expire. Releases go to the package registry or release assets.

TABLES

Table: rule · detail

Rule	Detail
Latest kept	Artifacts from the most recently completed pipeline and the most recent successful pipeline on each branch or tag are kept regardless of expire_in, until a newer pipeline replaces them (a project setting can turn this off).
Keep button	On a job page, Keep stops one job's artifacts from expiring.
Size	GitLab.com limits a job's artifacts to 1 GB.
untracked: true	adds every file Git does not track, handy for build folders, risky for secrets.

Lesson: Keeping files: the artifacts keyword

Table: setting on the job · downloads

Setting on the job	Downloads
nothing	artifacts of all jobs in earlier stages
`dependencies: [build]`	only build's artifacts (build must be in an earlier stage)
`dependencies: []`	nothing, the job starts faster
`needs: [build]`	only build's artifacts, and the job starts as soon as build is done
`needs: [{job: build, artifacts: false}]`	waits for build but downloads nothing
`needs: project: OWNER/other, job: build, ref: main, artifacts: true`	artifacts of the latest successful build job on main of another project, up to five such jobs (Premium and Ultimate)

Lesson: Passing artifacts between jobs

Table: from · where to click

From	Where to click
A pipelines list	Download artifacts, then pick the job
A jobs list	Download artifacts next to the job
A job page	Download or Browse in the Job artifacts panel
A merge request	Artifacts on the Overview page, for the latest pipeline
Build, Artifacts	every artifact in the project, with Download and Browse

Lesson: Pulling artifacts: UI, API, glab and job token

Table: rule · detail

Rule	Detail
ref_name	a branch or tag name. For merge request pipelines use refs/merge-requests/IID/head, the variable CI_MERGE_REQUEST_REF_PATH holds it.
Reports	not in the archive. Use the job id endpoint with file_type=junit, or list the file under paths.
Job token	the other project must add the calling project to its CI/CD job token allowlist (Settings, CI/CD, Job token permissions).
--location	GitLab.com may redirect through a CDN, so curl must follow redirects.

Lesson: Pulling artifacts: UI, API, glab and job token

Table: · artifacts · cache

	Artifacts	Cache
Meant for	results to pass on and keep	dependencies to reuse
Shared between	jobs of the pipeline, people, API, other projects	runs of jobs on the same runner or shared cache storage
Guaranteed	yes, until expiry	no, it is a best effort speed up
Keyword	`artifacts:`	`cache:`

Lesson: Caching on GitLab

TIPS AND TRAPS

COMMON MISTAKE reports such as junit are not inside the downloadable archive. To keep the XML as a file too, also list it under paths, as the sample project does.

Lesson: Keeping files: the artifacts keyword

IN THE EXAM The finished release belongs in the package registry or a release asset, not in a job artifact that may expire. For example, publish a Python wheel to the project's PyPI registry in a tag pipeline with the CI_JOB_TOKEN.

Lesson: Pulling artifacts: UI, API, glab and job token

STEP 9 Exam revision

9.1 Exam revision: commands, scenarios and pipeline bug hunts

Exam and interview style questions on Git and CI/CD with worked answers, a command table and pipeline bug hunts.

TABLES

Table: setting · typical format · what earns the marks · practise in

Setting	Typical format	What earns the marks	Practise in
College lab and semester exams	write the command, draw the commit graph, explain a term	the exact command and a correct graph	Lessons 2 and 3
Job interviews (Dev, QA, DevOps)	scenarios: what do you run, how do you design the pipeline	safe choices for shared history, clear reasons	Lessons 3 and 4
GitHub Actions and GitLab CI/CD certifications	multiple choice on syntax, triggers, artifacts, security	knowing defaults and limits exactly	Lesson 4 and the quiz
Code review at work	spot the bug in a workflow file	reading YAML carefully	Lesson 4

Lesson: How Git and CI/CD are examined

Table: task · command

Task	Command
Start a repository with main	`git init -b main`
Stage parts of a file	`git add -p`
See staged changes	`git diff --staged`
History as a graph	`git log --oneline --graph --all`
Who changed this line	`git blame -L 10,20 file`
Create and switch branch	`git switch -c name`
Merge keeping a merge commit	`git merge --no-ff name`
Abort a conflicted merge	`git merge --abort`
Replay branch on main	`git rebase main`
Clean up last 3 commits	`git rebase -i HEAD~3`
Copy one commit here	`git cherry-pick -x id`
Unstage	`git restore --staged file`
Undo last local commit, keep changes	`git reset --soft HEAD~1`
Undo a pushed commit	`git revert id`
Find lost commits	`git reflog`
Park work	`git stash push -u -m note`
Release tag	`git tag -a v1.0.0 -m msg` and `git push origin v1.0.0`
Publish a branch	`git push -u origin name`
Safe force push	`git push --force-with-lease`
Find the bad commit	`git bisect start, bad, good v1.0` or `bisect run`
Why is a file ignored	`git check-ignore -v path`
Object type and content	`git cat-file -t id`, `-p id`
Pull an Actions artifact	`gh run download RUN_ID -n name`
Pull a GitLab artifact	`glab job artifact main build` or the jobs/artifacts API

Lesson: Command recall in one table

TIPS AND TRAPS

IN THE EXAM When asked "how do you undo", first say where the mistake is (working tree, staging area, local commit, pushed commit) and whether others have it. The right command follows from those two facts.

Lesson: How Git and CI/CD are examined

REMEMBER In exams and reviews, read a pipeline file in this order: triggers, checkout or clone, versions, what each job needs from another job, what is kept, and what runs with which permissions.

Lesson: Spot the bug: pipeline files